

An algorithm for first order gradient based optimization of stochastic objective functions, based on adaptive estimates of lower order moments
straightforward to implement, computationally efficient and has little memory requirements

It is invariant to diagonal rescaling of the gradients

Well suited for problems that are large in terms of data and/or parameters

also appropriate for problems with non-stationary objectives and problems with very noisy / very sparse gradients

The base learning rate (α) is still global

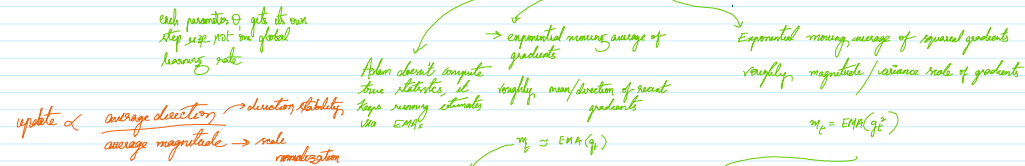
Adaptation happens through scaling by $\sqrt{v_t}$

Stochastic gradient based optimization is kind of method used to train machine learning models by gradually improving parameters using randomly sampled data instead of the entire dataset at once

The focus of this paper is on the optimization of stochastic objectives with high dimensional parameter spaces first order methods are used since higher order methods are ill suited

Adam ^(name is derived from adaptive moment estimator) → method for efficient stochastic optimization that only requires first order gradients with little memory requirements

The method computes individual adaptive learning rates for different parameters from estimates of first and second moments of gradients



variance first moment $\rightarrow$ low momentum $\rightarrow$ easy to get updates
variance second moment $\rightarrow$ no per parameter scaling $\rightarrow$ unstable when gradients vary across dimensions

Represents $\rightarrow$ first moment $\rightarrow$ direction / policy direction
second moment $\rightarrow$ size / prop / adapted step scaling

Adam = Momentum + RMSprop combined

Intuition - gradients point towards where the loss decreases
Individual gradients can be noisy
Averaging them gives a stable direction

where to go $\neq$ how large steps to take
squaring rates all values $\neq$ measures how large gradients tend to be
Intuition $\rightarrow$ large gradients $\rightarrow$ parameter changes too fast
small gradients $\rightarrow$ learning is slow
How big are gradients usually here?

It's harder to momentum - smoothing out cycles
Identify where the gradients have been pointing lately?

Scaling - extreme scale behavior

Large models $\rightarrow$ gradients vary wildly per parameter

Adaptation per parameter scaling becomes critical

At very large scale, second moment estimate can become stale or overly conservative

What problem is this fixing?

On a global learning rate fails when

$\rightarrow$ some parameters have large gradients

$\rightarrow$ others tiny gradients

Adam normalizes each parameters update automatically,

Gradients

Strong learning where $\rightarrow$ consistent gradient direction (big m_t)

$\rightarrow$ controlled variance (reasonable v_t)

$\rightarrow$ weak learning where gradients fluctuate or explode (large v_t slows updates)

RD1 (R) each parameter gets a small local optimization state (m_t, v_t)

(D) Still minimizing the same loss $\rightarrow$ Adam only changes the geometry of its step

(I) Gradients $\rightarrow$ compounded into bias EMAs $\rightarrow$ used to decide future step size

First moment = Mean

Second moment = uncentered variance proxy

(Adam uses $E(g)$ and not variance $E(g - \bar{g})^2$)

Adam $\rightarrow$ advantages

(1) Magnitudes of parameter updates are invariant to scaling of the gradient

If gradients are multiplied by a constant c
 $g_t \rightarrow c g_t$ then in Adam $m_t \propto c$ but update use $\frac{m_t}{\sqrt{v_t}}$ so $\frac{c}{\sqrt{c^2}} = 1$

update size stays roughly the same. Optimization is insensitive to gradient scale

(2) Step sizes are approximately bounded by step size hyperparameter

Base learning rate $\propto \alpha$ acts like a soft upper bound

Because Adam divides by $\sqrt{v_t}$ $\rightarrow$ updates never explode early $\rightarrow$ effective step is bounded by α

(3) doesn't require a stationary objective

distribution of gradients doesn't change over time

Adam works when $\rightarrow$ gradients change during training, loss landscape shifts

EMA Exponential Moving average

It is a way of computing running average that gives

more weight to recent gradients

less weight to older gradients

When training reaches gradients can be noisy

EMA helps by $\rightarrow$ smoothing out noise

(1) capture the general direction of recent gradients

(2) more consistent over time

Newest gradient $\rightarrow$ highest wt

Older gradients $\rightarrow$ less weight exponentially

$(1-p), (1-p)^2, \dots$

(i) doesn't require a stationary objective

distribution of gradients doesn't change over time

Adam works well → gradients change during training, loss landscape shifts

Because EMA gives more weight to recent gradients → adapt online

(iv) Works with sparse gradients

Adam keeps per parameter adaptive scaling
rarely updated parameters can still take meaningful steps

In contrast SGD requires manually tuning

(v) Naturally performs a form of step size annealing → learning rate effectively decreases over time

In Adam v_t → tend to grow or stabilize
denominator remains → update slower

Algorithm 1

g_t indicates element wise square $g_t \odot g_t$

default settings $\alpha = 0.001$ $\beta_1 = 0.9$ $\beta_2 = 0.999$ $\epsilon = 10^{-9}$

All operations on vectors are element wise

Require $\alpha \rightarrow$ stepsize

Require $\beta_1, \beta_2 \in (0, 1)$ exponential decay rates for the moment estimates

Require $f(\theta) \rightarrow$ stochastic Φ scalar function with parameters θ

Require $\theta_0 \rightarrow$ initial parameter vector

$M_0 \leftarrow 0$ (Initialize 1st moment vector)

$V_0 \leftarrow 0$ (Initialize 2nd moment vector)

$t \leftarrow 0$ (Initialize timestep)

while not converged do

$t \leftarrow t + 1$

$g_t \leftarrow \nabla_{\theta} f_t(\theta_{t-1})$ (get gradients w.r.t. stochastic objective at timestep t)

$M_t \leftarrow \beta_1 M_{t-1} + (1 - \beta_1) g_t$ (update biased first moment estimate)

$V_t \leftarrow \beta_2 V_{t-1} + (1 - \beta_2) g_t \odot g_t$ (update biased second raw moment estimate)

$\hat{M}_t \leftarrow M_t / (1 - \beta_1^t)$ (compute bias-corrected first moment estimate)

$\hat{V}_t \leftarrow V_t / (1 - \beta_2^t)$ (compute bias-corrected second raw moment estimate)

$\theta_t \leftarrow \theta_{t-1} - \alpha \hat{M}_t / (\sqrt{\hat{V}_t} + \epsilon)$ update parameters

end while

return θ_T (resulting parameters)

$f(\theta) \rightarrow$ noisy objective function Φ stochastic scalar function that is differentiable w.r.t. parameter θ

(we would like to minimize the expected value of this function)

$E[f(\theta)]$ w.r.t. θ

→ stochasticity may be from data points or inherent noise

$f_1(\theta), f_2(\theta) \rightarrow$ realizations of the stochastic function at subsequent time steps 1, 2

$g_t = \nabla_{\theta} f_t(\theta) \rightarrow$ gradient, i.e. the vector of partial derivatives of f_t w.r.t. θ evaluated at time step t

Also updates EMA's of gradient (M_t) and Squared gradient (V_t) hyperparameters $\beta_1, \beta_2 \in (0, 1)$ control the exponential decay rates of these moving averages

The moving averages themselves are estimates of the 1st moment (the mean) and the second raw moment (the uncentered variance of the gradient)

are initialized as (vector) 0's leading to moment estimates that are biased towards zero (especially during initial time steps and decay rates are small (β 's close to 1))

→ this initialization bias can be counteracted, resulting in bias-corrected estimates $\hat{M}_t$ and $\hat{V}_t$

β values are constants ($0.9, 0.999$) EXPONENTIAL MOVING AVERAGES

M_t, V_t (moving wps are) smoothly start at 0

increasing algo's efficiency by replacing last three lines in the loop with

$$d_t = \alpha \sqrt{1 - \beta_2^t} / (1 - \beta_1^t)$$

$$\theta_t \leftarrow \theta_{t-1} - \alpha_t M_t / (\sqrt{V_t} + \epsilon)$$

EMA helps by smoothing out noise

1) capture the general direction of recent gradients

2) make optimizations more stable

EMA formula

if g_t is gradient at step t
 $m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$ → smoothing factor (usually close to 1)

Key fact $E[g_t] = E[g^*]$ always true

$$\Rightarrow \frac{E[g_t]}{E[g^*]}$$

When many gradients are averaged, random fluctuations cancel out, no bias remains
cannot cancel RMS magnitude hence ϵ

geometric significance of $\frac{M_t}{V_t}$ if gradients point uniformly

mean is large
RMS is similar scale $\frac{M_t}{V_t} \approx 1$

→ strong, confident direction → large step

if gradients fluctuate

→ mean cancel out $\frac{M_t}{V_t} \approx 0$

→ RMS remains large

→ in certain direction, small steps

Near optimum gradient steps agreeing with each other

→ true gradient magnitude becomes small
→ stochastic gradients fluctuate due to noise

$\Rightarrow M_t \rightarrow 0 \Rightarrow V_t$ stays non-zero

$\frac{M_t}{V_t} \rightarrow 0 \Rightarrow \Delta_t$ becomes smaller automatically

First moment = mean
Second raw moment = uncentered variance
Raw moment

For a random variable

1st moment $E(x)$

avg. value (mean)

2nd moment $E(x^2)$

avg of squared values (called second raw moment)

→ because it is not centered around the mean

Raw second moment $\rightarrow E(x^2)$

Variance (centered) $\rightarrow E[(x - E(x))^2]$

$$\text{Var}(x) = E(x^2) - (E(x))^2$$

Adam stores only $E[g^*]$

First moment estimate $M_t \approx E[g]$

→ average direction

Second raw moment estimate

$$V_t \approx E[g^2]$$

→ average magnitude

Why Adam uses raw moment

→ if Adam used variance $E[g - E(g)]^2$

there $M_t = \text{signal (direction)}$

$V_t = \text{signal energy / power}$

it would require → storing more info
extra computation
instability due to subtraction

Adam divides signal by its energy scale

Raw moment → simple
→ always positive
→ directly useful for normalization

$$d_t = \alpha \frac{\sqrt{1-\beta_t^2}}{(1-\beta_t^2)} \\ \theta_t \leftarrow \theta_{t-1} - \alpha_t m_t / (\sqrt{V_t} + \epsilon)$$

paper is saying that bias correction can be folded into learning rate scaling
 * Same math, fewer operations

Why dividing by $\sqrt{(\text{second raw moment})}$ stabilizes optimization

large gradients $\rightarrow$ large $V_t \rightarrow$ large denominator $\rightarrow$ smaller step (stabilization comes from adaptive normalization, not from squaring alone)

Adap tries to scale normalization across parameters (it makes updates comparable even if gradient magnitudes differ)

denominator estimates typical gradient size $\rightarrow$ divide by it $\rightarrow$ step becomes scale invariant

if gradients are mostly big $\rightarrow$ shrink step
 " " " " small $\rightarrow$ enlarge relative step

2.1 Adam's update rule

important property $\rightarrow$ careful choice of step size

Assume $\epsilon=0$ the effective step taken in parameter space at a timestep t is $\Delta_t = \alpha \hat{m}_t / \sqrt{V_t}$

has two upper bounds $|\Delta_t| \leq \alpha (1-\beta_1) / \sqrt{1-\beta_2}$

$$(1-\beta_1) \geq \sqrt{1-\beta_2} \text{ and } |\Delta_t| \leq \alpha$$

happens in most cases
 one of sparsity
 when a gradient has
 been zero at all steps
 excepts around step

for less sparse cases, the effective step size will be smaller

When $(1-\beta_1) = \sqrt{1-\beta_2}$ we have $|\hat{m}_t|/\sqrt{V_t} \leq 1$ therefore $|\Delta_t| \leq \alpha$

In most common scenarios, we will have that $|\hat{m}_t|/\sqrt{V_t} \leq 1$ since $E[|\sqrt{E[g_t^2]}|] \leq 2$

The effective magnitude of steps taken in parameter space at each time step are approximately bounded by step size, setting $\alpha = 1$ $|\Delta_t| \leq \alpha$

We call the ratio $m_t/\sqrt{V_t}$ Signal to noise ratio (SNR)

With smaller SNR effective step size Δ_t will be closer to zero

there is quite uncertainty about whether the direction of $\hat{m}_t$ corresponds to the direction of true gradient

2.1 says Adam's normalized update cannot become arbitrarily large in sparse cases it is bound by a term reducing β_1 and normal case it is bounded by α

3 Initialization Bias Correction

deriving bias term for second raw moment (unscaled variance) using EMA of squared gradient

$\beta_2 \rightarrow$ decay rate

$g_1, g_2 \rightarrow$ gradients at subsequent time steps (from a gradient chain $g_t \sim p(g_{t-1})$)

$V_0=0$ (initialize)

note $V_t = \beta_2 V_{t-1} + (1-\beta_2) g_t^2$ can be written as $V_t = (1-\beta_2) \sum_{i=1}^t \beta_2^{t-i} g_i^2$ (1)

how $E[V_t]$ the expected value of EMA at t relates to true second moment $E[g_t^2]$

$$E[V_t] = E \left[(1-\beta_2) \sum_{i=1}^t \beta_2^{t-i} g_i^2 \right] \quad (2)$$

first moment estimate $m_t \approx E[g_t]$

$\rightarrow$ average direction

Second raw moment estimate

$$V_t \approx E[g_t^2]$$

$\rightarrow$ average squared magnitude

full rewrite

Step 1 $\rightarrow$ Adam's update (ignore ϵ)

$\Delta_t = \alpha \frac{\hat{m}_t}{\sqrt{V_t}}$ $\rightarrow$ bias corrected first moment

$\Delta_t = \alpha \frac{\hat{m}_t}{\sqrt{V_t}}$ $\rightarrow$ bias corrected second moment

Step 2 $\rightarrow$ Bias correction terms

$$\hat{m}_t = \frac{m_t}{1-\beta_1} \quad \hat{V}_t = \frac{V_t}{1-\beta_2}$$

Step 3 $\rightarrow$ effective magnitude

$$|\Delta_t| = \alpha \frac{|\hat{m}_t|}{\sqrt{\hat{V}_t}} = \alpha \frac{\frac{|\hat{m}_t|}{1-\beta_1}}{\sqrt{\frac{V_t}{1-\beta_2}}}$$

Step 4 $\rightarrow$ worst case upper bound

In sparse gradient case

$\rightarrow$ only one nonzero gradient sample dominates

$$\text{then } \frac{|\hat{m}_t|}{\sqrt{\hat{V}_t}} \approx \frac{1-\beta_1}{\sqrt{1-\beta_2}}$$

$$\text{so } |\Delta_t| \leq \alpha \frac{(1-\beta_1)}{\sqrt{1-\beta_2}}$$

Steps $\rightarrow$ common case bound (usually gradients aren't sparse)

$$\frac{m_t}{\sqrt{V_t}} \approx \frac{E[g_t]}{\sqrt{E[g_t^2]}} \leq 1 \quad \text{if } |\Delta_t| \leq \alpha \quad \text{step size roughly bounded by learning rate}$$

* In sparse updates the moving averages are dominated by current gradient sample

(2) $\rightarrow$ (3) $\rightarrow$ (4) 1

Goal: compare EMA to true moment

Eq 2 just applying expectation $E[V_t] = E[(1-\beta_2) \sum \beta_2^{t-i} g_i^2]$

Eq 3 (since expectation is linear, pull constants outside)

$$= (1-\beta_2) \sum \beta_2^{t-i} E[g_i^2]$$

They assume $E[g_t^2] \approx E[g_0^2]$

i.e second moment roughly stationary

$$\text{Then } E[V_t] = E[g_0^2] (1-\beta_2) \sum \beta_2^{t-i} + \zeta \rightarrow \text{small correction}$$

Now use geometric series

$$\sum_{i=1}^t \beta_2^{t-i} = \frac{1-\beta_2^t}{1-\beta_2}$$

Multiply by β_2

$$E[V_t] = E[g_0^2] [1-\beta_2^t] + \zeta$$

Because $V_0=0$

$$|\Delta_t| = \alpha \frac{|\hat{m}_t|}{\sqrt{\hat{V}_t}} = \alpha \frac{\frac{|\hat{m}_t|}{1-\beta_1}}{\sqrt{\frac{V_t}{1-\beta_2}}}$$

for sparse case $m_t \approx (1-\beta_1)g_t$

$$V_t \approx (1-\beta_2)g_t^2$$

$$\frac{|\hat{m}_t|}{\sqrt{V_t}} \approx \frac{1-\beta_1}{\sqrt{1-\beta_2}} \Rightarrow |\Delta_t| \approx \alpha \frac{(1-\beta_1)}{\sqrt{1-\beta_2}} \frac{\sqrt{1-\beta_2}}{1-\beta_1}$$

(The bound is tighter at early time steps so effectively $t \approx 1$)

So the second fraction canceled out first

$$\frac{1-\beta_1}{\sqrt{1-\beta_2}} \frac{\sqrt{1-\beta_2}}{1-\beta_1} = 1$$

the final bound becomes $|\Delta_t| \leq \alpha \frac{1-\beta_1}{\sqrt{1-\beta_2}}$

Section 3.1 previous

Adam automatically keeps parameter steps controlled

Two scenarios $\rightarrow$ sparse gradients $\rightarrow$ special bound with β_1 term

$\rightarrow$ normal gradients $\rightarrow$ step is bounded by α

$$\begin{aligned}
 E[V_t] &= E \left[(1-\beta_2) \sum_{i=1}^t \beta_2^{t-i} g_i \right] \quad (2) \\
 &= E[g_t] (1-\beta_2) \sum_{i=1}^t \beta_2^{t-i} + \zeta \quad (3) \\
 &= E[g_t] (1-\beta_2) + \zeta \quad (4)
 \end{aligned}$$

$\zeta = 0$ if the true second moment E is stationary

otherwise it can be kept small since β_2 can be chosen such that EMA averages smaller into the gradient too far in the past

What is the bias term $(1-\beta_2)$ which is caused by initializing the running average to zero, in algorithm 1, we therefore divide by the term to correct the initialization bias

In case of sparse gradients for a reliable estimate of the second moment one needs to average over many gradients by choosing a small value of β_2

however this case where a bias of initialization bias correction would lead to initial steps that are much larger

4 Convergence analysis

given an arbitrary, unknown sequence of convex cost functions $f_1(\theta), f_2(\theta), \dots, f_T(\theta)$. At each time t our goal is to predict the parameter θ_t^* and evaluate it on a possibly unknown cost function f_t

value of sequence decreases as advances

we evaluate the algorithm using the regret $\rightarrow$ sum of all past differences between online prediction $f_t(\theta_t)$ and best fixed past parameter $f_t(\theta^*)$ from a feasible set X for all the previous steps

$$R(T) = \sum_{t=1}^T (f_t(\theta_t) - f_t(\theta^*))$$

where $\theta^* = \arg \min_{\theta \in X} \sum_{t=1}^T f_t(\theta)$ we show Adam has $O(\sqrt{T})$ regret bound

$g_t \triangleq \nabla f_t(\theta_t)$, $g_{t,i}$ as the i -th element

$g_{t,i} \in \mathbb{R}^d$ as a vector that contains the i -th dimension of the gradients overall iterations till t , $g_{t,i} = [g_{1,i}, g_{2,i}, \dots, g_{t,i}]$

$$\gamma \triangleq \frac{\beta_1}{\sqrt{\beta_2}}$$

following theorem holds the learning rate α_t is decaying at rate of $T^{-1/2}$ and first moment running average coefficient β_1 decay exponentially with λ , that is typically close to 1 eg $1-10^{-5}$

Theorem 4.1 $\rightarrow$ Assume that the function f_t has bounded gradients $\|\nabla f_t(\theta)\|_2 \leq G_1$, $\|\nabla f_t(\theta)\|_\infty \leq G_\infty$ for all $\theta \in \mathbb{R}^d$ and distance between any θ_t generated by Adam is bounded $\|\theta_n - \theta_m\|_2 \leq D$, $\|\theta_n - \theta_m\|_\infty \leq D_\infty$ for any $m, n \in \{1, \dots, T\}$, $\beta_1, \beta_2 \in (0, 1)$ satisfy $\frac{\beta_1}{\beta_2} < 1$

Let $\alpha_t = \frac{\alpha}{\sqrt{t}}$ and $\beta_{1,t} = \beta_1 \lambda^{t-1}$, $\lambda \in (0, 1)$. Adam achieves the following guarantee, for all $T \geq 1$

$$R(T) \leq \frac{D^2}{2\alpha(1-\beta_2)} \frac{1}{\sqrt{1-\beta_2}} \sqrt{T} + \frac{\alpha(1+\beta_1)G_\infty}{(1-\beta_1)\sqrt{1-\beta_2}(1-\lambda)^2} \frac{1}{\sqrt{1-\beta_2}} \sqrt{T} + \frac{1}{\sqrt{1-\beta_2}} \frac{D_\infty G_1 \sqrt{1-\beta_2}}{2\alpha(1-\beta_1)(1-\lambda)^2}$$

Theorem implies when data features are sparse and bounded gradients

the summation term can be much smaller than its upper bound $\sum_{i=1}^t \|g_{i,T}\|_2 \ll d G_\infty \sqrt{T}$ and $\sum_{i=1}^t \sqrt{T} \ll d G_1 \sqrt{T}$ in particular if the class of the functions and data features are in the form of section 1.2 in (Duchi et al 2011)

These results for the expected value also apply to Adam

$$E \left[\sum_{i=1}^t \|g_{i,T}\|_2 \right]$$

In particular, the adaptive method, such as Adam & Adagrad, can achieve $O(\log \sqrt{T})$ an improvement over $O(\sqrt{T})$ for non adaptive method

Decaying $\beta_{1,t}$ towards zero is equivalent to reducing momentum coefficient in the end of training can improve convergence

Corollary 4.2

Assume that the function f_t has bounded gradients $\|\nabla f_t(\theta)\|_2 \leq G_1$, $\|\nabla f_t(\theta)\|_\infty \leq G_\infty$ for all $\theta \in \mathbb{R}^d$ and distance between any θ_t generated by Adam is bounded $\|\theta_n - \theta_m\|_2 \leq D$, $\|\theta_n - \theta_m\|_\infty \leq D_\infty$ for any $m, n \in \{1, \dots, T\}$

Adam achieves the following guarantee for all $T \geq 1$

$$\frac{R(T)}{T} = O \left(\frac{1}{\sqrt{T}} \right)$$

This can be obtained by using Theorem 4.1 and $\sum_{i=1}^t \|g_{i,T}\|_2 \leq d G_\infty \sqrt{T}$ Thus $\lim_{T \rightarrow \infty} \frac{R(T)}{T} = 0$

"They tried to prove" if we pretend training is an online convex optimization problem, then Adam has a bound regret

$\rightarrow$ A theoretical safety guarantee

$\leq G_0$

Monotonic

Adagrad

SGD $\rightarrow$ update

$\theta = \theta_t - \alpha g_t$

same learning rate for all parameters

continuous, the gradient scale

$\eta_t = 1 / (1 + \dots)$

AdaGrad

key idea

decide by cumulative gradients

squared

$\frac{g_t}{\sqrt{1 + \dots}}$

RMSProp

fixes Adagrad

key changes instead of cumulative averages

$\rightarrow$ EMA of squared gradients

Adaptive learning rate

Adam clamps

combines

Momentum (first moment)

RMS prop (second moment)

Other Methods briefly mentioned

AdaDelta / SGDR / Natural Gradient

adaptive step size estimations

curvature ideas

Adaptive learning rate

Substitute V_{t-1}

$$= \beta_2 (V_{t-1} + (1-\beta_2) g_{t-1}^2) + (1-\beta_2) g_t^2$$

expand

$$= \beta_2 V_{t-1} + (1-\beta_2) \beta_2 g_{t-1}^2 + (1-\beta_2) g_t^2$$

Keep expanding to constant

$$V_t = (1-\beta_2) \sum_{i=1}^t \beta_2^{t-i} g_i^2 \quad (\text{older gradients get powers of } \beta_2)$$

$\rightarrow$ exponentially smaller not

$$\lim_{t \rightarrow \infty} V_t = 1 - \beta_2$$

Multiply by β_2

$$E[V_t] = E[g_t^2] (1-\beta_2) + \zeta$$

$\rightarrow$ Because $V_0 = 0$

$$E[V_t] = E[g_t^2] (1-\beta_2)$$

Early training $1-\beta_2 < 1$

So EMA underestimates true second moment

This is the initialization bias

To fix underestimation

$$\hat{V}_t = \frac{V_t}{1-\beta_2^t}$$

Now $E[\hat{V}_t] \approx E[g_t^2]$

Bias removed

Sparse case

if gradients are sparse

need small β_2 (first moment)

But small $\beta_2 \rightarrow$ strong initialization bias early

(hence correction is important)

First Moment Derivation

$$m_t = \beta_1 v_{t-1} + (1-\beta_1) g_t$$

$$m_t = (1-\beta_1) \sum_{i=1}^t g_i$$

$$E[m_t] = E[g] (1-\beta_1^t)$$

$$\text{Bias correction} = m_t = \frac{m_t}{1-\beta_1^t}$$

Bias correction exists because

EMA starts from zero, so early estimates are systematically too small

Why exactly stronger at early time steps?

EMA behaves as if it has seen fewer samples

Early time steps

$\rightarrow$ only a few gradients contribute

$\rightarrow$ but with are normalized as if history was long

* clean definition

Bias is strongest at early time steps because the EMA starts at zero and has accumulated only a few gradient samples, so the running average underestimates the true moment by a factor of $(1-\beta_1^t)$

Intuition

EMA averages infinite history

But at step 1 the history is short

so estimate is optimally too small

Bias correction fixes this mismatch

SGD

Momentum

AdaGrad

RMSProp

Adam

Adam = Momentum + RMSprop + bias correction

SGD → update

$\theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{L}$

same learning rate for all parameters

scalable, the product scale

Momentum (Polyak)

idea smooth gradient direction over time

Mechanism: sum of gradients

it solves noisy oscillations

doesn't solve adaptive parameter scaling

AdaGrad

key idea

divide by cumulative squared

gradients

Meaning: frequently updated parameters slow down

rare features get larger updates

problem: learning rate keeps shrinking forever

training eventually stops moving

Fixed AdaGrad

key change

instead of cumulative averages

→ EMA of squared gradients

Adaptive scaling preserved

learning rate doesn't decay forever

problem: No momentum

No bias correction

Adam clanks

combines

Momentum (first moment)

RMSprop (second moment)

Bias correction (Early stability)

1/ Adam

causes momentum → you get RMSprop

removes normalization → you get momentum

removes both → we get SGD

Other Methods briefly mentioned

AdaDelta / RMSProp / Natural Gradient

adaptive step size estimation

convergence ideas

Adam belongs to this general family

Adaptive first-order methods

SFO (sum of functions optimizers)

Quasi-Newton style

Requires large memory

Proper motivation to robust scalability

Adam behaves like conservative diagonal preconditioning

Each parameter scaled separately

Approximation of second order information

expressions show

Adam converges fast

works across many models

robust to hyper parameters

Main claims

good default learning rate

Strong performance on sparse gradients

low tuning effort

if you have to back up math

norm = a way to measure size/length of vector

$L_2 \text{ norm} = \|x\|_2 = \sqrt{x_1^2 + x_2^2 + \dots + x_n^2}$ (Standard Euclidean length)

$L_1 \text{ norm} = \|x\|_1 = |x_1| + |x_2| + \dots + |x_n|$ Manually added absolute magnitude

$L_\infty \text{ norm} = \|x\|_\infty = \max |x_i|$

largest absolute component

only the biggest value matters

eg $x = [-4, 2, 1]$ $L_1 \text{ norm} \rightarrow$ considers everything (smooth averaging)

$L_\infty \text{ norm} \rightarrow$ just looks at +

Adamax → normalization based on largest gradient component instead of average magnitude

L_2 style norm → even about avg magnitude sensitive to all components

L_∞ Scaling → dominated by largest gradient component more stable in some cases

Arb/Max

in adam, the update rule for individual weights also scale their gradients separately, proportional to a (scaled) L^p norm of their individual current and past gradients

we can change from L^2 norm based update rule to a L^p norm based update rule

they become numerically unstable for large 'p'

However if $p \rightarrow \infty \rightarrow$ stable

Let a case of L^p norm, the step size at time t be inversely proportional to v_t^p

$$\text{where } v_t = \beta_1^p v_{t-1} + (1 - \beta_1^p) |g_t|^p \\ = (1 - \beta_1^p) \sum_{i=1}^t \beta_1^{p(t-i)} |g_i|^p$$

Algorithm 2 Adamax $\alpha \leftarrow 0.001, \beta_1 \leftarrow 0.9, \beta_2 \leftarrow 0.999, \beta_1^t \leftarrow \alpha / (1 - \beta_1^t)$ is the learning rate with the bias correction then for the first moment

Require: d stepsize

Require: $\beta_1, \beta_2 \in [0, 1]$

Require: $f(\theta)$

Require: θ_0 Initial parameter vector

$m_0 \leftarrow 0$ (initialize first moment vector)

$v_0 \leftarrow 0$ (initialize exponentially weighted infinity norm)

$t \leftarrow 0$ (initialize time step)

While θ_t not converged do

$t \leftarrow t + 1$

$g_t \leftarrow \nabla_{\theta} f(\theta_{t-1})$ (get gradients w.r.t. stochastic objective at time step t)

$m_t \leftarrow \beta_1 m_{t-1} + (1 - \beta_1) g_t$ (update biased first moment estimate)

$v_t \leftarrow \max(\beta_2 v_{t-1}, |g_t|)$ (update the exponentially weighted infinity norm)

$\theta_t \leftarrow \theta_{t-1} - (\alpha / (1 - \beta_1^t)) m_t / v_t$ (update parameters)

end while

return θ_t (resulting parameter)

what's changing from Adam to Adamax

→ this approximates $E|g|$

in adam $v_t = \beta_2 v_{t-1} + (1 - \beta_2) |g_t|$ update uses $\frac{\eta g_t}{\sqrt{v_t}}$

This is essentially an L_2 normalization

in Adamax L_∞ Scaling instead of L^2 norm

for general $v_t = \beta_1^p v_{t-1} + (1 - \beta_1^p) |g_t|^p$

$v_t = \max(\beta_2 v_{t-1}, |g_t|)$

define $u_t = v_t^{1/p}$

then limit $p \rightarrow \infty$

$\rightarrow L_\infty$ norm

$\lim_{p \rightarrow \infty} (\sum |x_i|^p)^{1/p} = \max |x_i|$

$\rightarrow$ maximum component

trying to prove p

taking $p \rightarrow \infty$

only largest term survives

that is why it becomes more

Adam denominator $\sqrt{v_t}$

Adamax $\rightarrow u_t$ numerator

why step size bounded by α ?

in Adamax

Why L_∞ norm is more stable than L_2 normalization in very high dimensions

L_2 norm aggregates all components $\|g\|_2 = \sqrt{\sum g_i^2}$

if many components are moderately large

L_2 grows with dimensions

L_∞ ignores growth $\|g\|_\infty = \max |g_i|$

only depends on largest component

In high dimensional spaces

L_2 norm can grow due to small contributions

L_∞ norm stays controlled by worst component

L_∞ norm is less sensitive to dimensionality

gives a more conservative, stable denominator

why conservative

L_∞ always needs to largest component

$$\theta_t \leftarrow \theta_{t-1} - (\alpha/(1-\beta^t)) m_t \quad (\text{update parameter})$$

and while
return θ_t (returning parameter)

Note that the decay term has equivalently been bounded β_t^2 instead of β_t . Now let $p \rightarrow \infty$, define $u_t = \lim_{p \rightarrow \infty} (v_t)^{1/p}$ then

$$\begin{aligned} u_t &= \lim_{p \rightarrow \infty} (v_t)^{1/p} = \lim_{p \rightarrow \infty} \left((1-\beta^t) \sum_{i=1}^t \beta^{(t-i)p} |g_i|^p \right)^{1/p} \\ &= \lim_{p \rightarrow \infty} (1-\beta^t)^{1/p} \left(\sum_{i=1}^t \beta^{(t-i)p} |g_i|^p \right)^{1/p} \\ &= \lim_{p \rightarrow \infty} \left(\sum_{i=1}^t \beta^{(t-i)p} |g_i|^p \right)^{1/p} \\ &= \max(\beta_t^{t-1} |g_1|, \beta_t^{t-2} |g_2|, \dots, \beta_t |g_{t-1}|, |g_t|) \end{aligned}$$

Which corresponds to recursively sample recursion formula

$$u_t = \max(\beta_t u_{t-1}, |g_t|)$$

With initial value $u_0 = 0$
we don't need to correct for initialization bias in this case

Magnitude of parameter updates has a simple bound with Adamw than Adam $|\Delta \theta_t| \leq \alpha$

Temporal averaging

Since the last iterate is noisy due to stochastic approximation, better generalization performance is achieved by averaging EMAs over the parameter can be used giving higher wt to the more recent parameter values

This can be heuristically implemented by adding one more line to the main loop of above algorithm

This can be heuristically implemented by adding one more line to the main loop of above algorithm

$$\bar{\theta}_t \leftarrow \beta_t \bar{\theta}_{t-1} + (1-\beta_t) \theta_t$$

with $\bar{\theta}_0 = 0$

Initialization bias can again be corrected by the estimator $\hat{\theta}_t = \bar{\theta}_t / (1-\beta_t^t)$

Conclusion

A simple and computationally efficient algorithm for gradient-based optimization of stochastic objective function

Aimed at ML problems with large dataset and/or high dimensional parameter space

Combines the advantages of RMSProp to deal with sparse gradients

RMSProp's ability to deal with non-stationary objectives

Symbols

$\triangleq$ "is defined as" or "equal by definition"

eg $A \triangleq B$ means A is defined as B

$\nabla \rightarrow$ nabla

eg $\nabla f(\theta_t)$ gradient

$\|x\|_2 = L_2 \text{ norm} = \text{vector length}$

$\|x\|_\infty = \text{max absolute component}$ } bounded norm

What is regret?

At each time step t , nature gives a loss $f_t(\theta)$
you choose parameter θ_t

you suffer loss $f_t(\theta_t)$

Imagine a system knowing future and fixed the best parameter θ^*

Regret how much worse you did compared to that best fixed choice

Equation n paper $R(T) = \sum_{t=1}^T (f_t(\theta_t) - f_t(\theta^*))$ Meaning = cumulative extra loss

that is why it becomes more

Adam denominator $\sqrt{v_t}$?
Adamw $\rightarrow u_t$ simpler

u_t is just a running minimum (~with decay)

No square root

No squared gradients

No bias correction needed

Why no bias correction?

In Adam $u_t \approx \mathbb{E}[g^t] (1-\beta^t)$

Because EMA accumulates slowly $\rightarrow$ biased low early

In Adamw $u_t = \max(\beta_t u_{t-1}, |g_t|)$

Thus no averaging

If $u_t = 0$ then first step $u_t = |g_t|$

No systematic shrinking $\rightarrow$ no bias needed

Temporal averaging $\rightarrow$ about averaging parameters

polyak averaging

instead of final parameter = last-iterate,

$$\text{use } \bar{\theta} = \frac{1}{T} \sum_{t=1}^T \theta_t$$

Reduces variance

works regardless of SGD

why step size bounded by α ?

In Adamw

$$\theta_{t+1} = \theta_t - \alpha \frac{\hat{m}_t}{u_t}$$

Since

$$u_t \geq |g_t|$$

And

$$|\hat{m}_t| \leq |g_t|$$

trivially bounded by gradient magnitude

So $|\Delta \theta_t| \leq \alpha$

Clearer bound than Adam

Adam normalizes by RMS magnitude
avg magnitude method

} Smooth normalization

Adamw normalizes by worst-case magnitude
dominated by largest component

} Conservative normalization

May prefer

$$\text{EMA of params } \theta_t^{\text{EMA}} = \beta \theta_{t-1}^{\text{EMA}} + (1-\beta) \theta_t$$

and α lin in training loop

this helps because gradients are noisy
parameters oscillate
averaging/smooths trajectory

improves generalization

gives a more conservative, stable denominator

why conservative

$\hookrightarrow$ always needs to largest component

if a component or coordinate is very large, unrepresentative and

normalization that divides all components by it

so components don't need such heavy damping

Bias correction in parameter averaging

has EMA start at 0

$$\theta_t^{\text{EMA}} / (1-\beta^t) \text{ removes bias}$$

Regret how much worse you did compared to that best fixed choice

Equation in paper $R(T) = \sum_{t=1}^T (L_t(\theta_t) - L_t(\theta^*))$ Meaning = cumulative extra loss

$\theta^* = \arg \min_{\theta \in \chi} \sum_{t=1}^T L_t(\theta)$ θ^* is the parameter value that minimises total loss over all timesteps

common $\Rightarrow$ value of θ that minimises

$\chi \rightarrow$ feasible set (allowed parameter space)

$R(T) = O(\sqrt{T})$ (regret grows slowly compared to time)

Average regret $R(T)/T \rightarrow 0$

interpretation: Algorithm becomes nearly as good as the best fixed parameter

Since Regret grows sublinearly ($\sqrt{T}$) average regret goes to zero, meaning the algorithm's per step performance approaches that of the best fixed parameter in hindsight